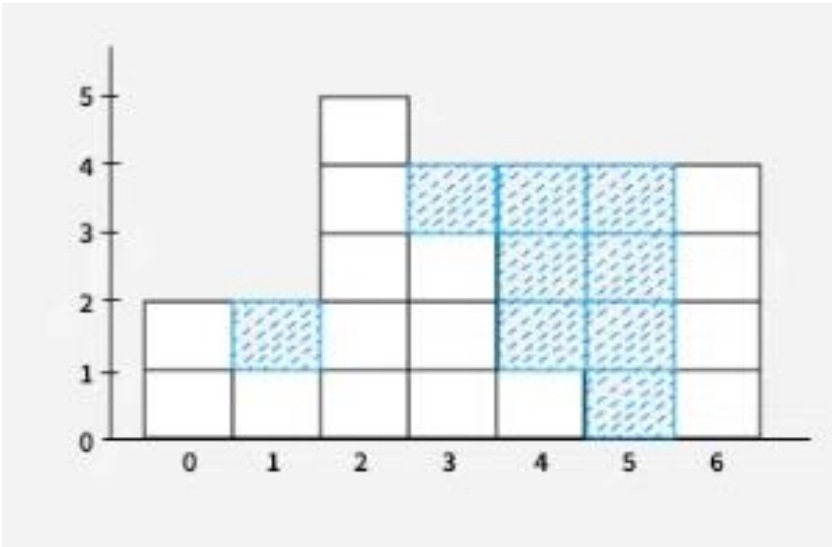
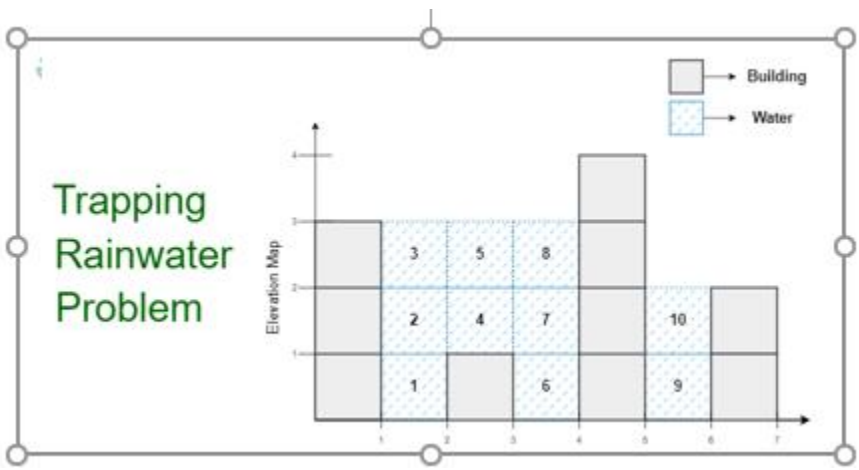
Illustration of Trapped Rain Water Problem Using Prefix Max & Suffix Max:

Sample Elevations:




```
int[] arr = {2, 1, 5, 3, 1, 0, 4};
```




```
int[] arr = {3, 0, 1, 0, 4, 0, 2};
```

Solution:

🧱 Step 1: Build `prefixMax` array

🧱 Step 2: Build `suffixMax` array

💧 Step 3: Calculate trapped water at each index:

```
waterAtI = Math.min(prefixMax[i], suffixMax[i]) - height[i];
```

```java
public class TrappingRainWater {

    // Function to compute trapped rainwater
    public static int trap(int[] height) {

        if (height == null || height.length < 3) return 0;

        int n = height.length;
        int[] prefixMax = new int[n];
        int[] suffixMax = new int[n];
        int trappedWater = 0;

        // 1. Build prefixMax array
        prefixMax[0] = height[0];
        for (int i = 1; i < n; i++) {
            prefixMax[i] = Math.max(prefixMax[i - 1], height[i]);
        }

        // 2. Build suffixMax array
        suffixMax[n - 1] = height[n - 1];
        for (int i = n - 2; i >= 0; i--) {
            suffixMax[i] = Math.max(suffixMax[i + 1], height[i]);
        }

        // 3. Calculate trapped water
        for (int i = 0; i < n; i++) {
            int waterAtI = Math.min(prefixMax[i], suffixMax[i]) - height[i];
            if (waterAtI > 0) {
                trappedWater += waterAtI;
            }
        }

        return trappedWater;
    }

    // Main method to run test cases
    public static void main(String[] args) {
        int[][] testCases = {
            {0,1,0,2,1,0,1,3,2,1,2,1},  // Expected Output: 6
            {3, 0, 1, 0, 4, 0, 2},      // Expected Output: 10
            {3, 0, 2, 0, 4},            // Expected Output: 7
            {1, 2, 3, 4},               // Expected Output: 0
            {2, 1, 5, 3, 1, 0, 4}       // Expected Output: 9
        };

        for (int i = 0; i < testCases.length; i++) {
```

```
            int result = trap(testCases[i]);
            System.out.println("Test Case " + (i + 1) + ": Trapped Water = "
+ result);
        }
    }
}
```

Test Cases

1. **Test Case 1**: {0,1,0,2,1,0,1,3,2,1,2,1}
    - Water is trapped between the bars, totaling **6 units**.
2. **Test Case 2**: {3, 0, 1, 0, 4, 0, 2}
    - Significant trapping occurs between the peaks, totaling **10 units**.
3. **Test Case 3**: {3, 0, 2, 0, 4}
    - Water is trapped in the valleys, totaling **7 units**.
4. **Test Case 4**: {1, 2, 3, 4}
    - No trapping possible as the bars are in ascending order, totaling **0 units**.
5. **Test Case 5**: {2, 1, 5, 3, 1, 0, 4}
    - Multiple valleys lead to trapping, totaling **9 units**.

📘 Test Case 1: {0,1,0,2,1,0,1,3,2,1,2,1}

Index (i)	height[i]	prefixMax[i]	suffixMax[i]	min(prefix, suffix)	waterAtI = min - height[i]
0	0	0	3	0	**0**
1	1	1	3	1	**0**
2	0	1	3	1	**1**
3	2	2	3	2	**0**
4	1	2	3	2	**1**
5	0	2	3	2	**2**
6	1	2	3	2	**1**
7	3	3	3	3	**0**
8	2	3	2	2	**0**
9	1	3	2	2	**1**
10	2	3	2	2	**0**
11	1	3	1	1	**0**

Total Trapped Water: 1 (i=2) + 1 (i=4) + 2 (i=5) + 1 (i=6) + 1 (i=9) = **6**

📘 Test Case 2: {3, 0, 1, 0, 4, 0, 2}

Index (i)	height[i]	prefixMax[i]	suffixMax[i]	min(prefix, suffix)	waterAtI = min - height[i]
0	3	3	4	3	0
1	0	3	4	3	3
2	1	3	4	3	2
3	0	3	4	3	3
4	4	4	4	4	0
5	0	4	2	2	2
6	2	4	2	2	0

Total Trapped Water: 3 (i=1) + 2 (i=2) + 3 (i=3) + 2 (i=5) = **10**

📘 Test Case 3: {3, 0, 2, 0, 4}

Index (i)	height[i]	prefixMax[i]	suffixMax[i]	min(prefix, suffix)	waterAtI = min - height[i]
0	3	3	4	3	0
1	0	3	4	3	3
2	2	3	4	3	1
3	0	3	4	3	3
4	4	4	4	4	0

Total Trapped Water: 3 (i=1) + 1 (i=2) + 3 (i=3) = **7**

📘 Test Case 4: {2, 1, 5, 3, 1, 0, 4}

Index (i)	height[i]	prefixMax[i]	suffixMax[i]	min(prefix, suffix)	waterAtI = min - height[i]
0	2	2	5	2	0
1	1	2	5	2	1
2	5	5	5	5	0
3	3	5	4	4	1
4	1	5	4	4	3
5	0	5	4	4	4
6	4	5	4	4	0

Total Trapped Water: 1 (i=1) + 1 (i=3) + 3 (i=4) + 4 (i=5) = **9**